

Modul 12

Membuat Aplikasi React Blog Post Menggunakan API



Oleh:

Nama:

Fadhilah Azzahra

NIM:

707012500118

Kelas:

D4SIKC-49-04

Mata Kuliah:

Pemrograman Web

PROGRAM STUDI S.Ter SISTEM INFORMASI KOTA CERDAS

FAKULTAS ILMU TERAPAN

UNIVERSITAS TELKOM

BANDUNG

2026

1. Tujuan

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- 1) Memahami konsep dasar React sebagai library JavaScript untuk membangun antarmuka pengguna (UI) berbasis component dan declarative programming.
- 2) Memahami arsitektur UI berbasis component serta perbedaan pendekatan React dibandingkan manipulasi DOM tradisional (Vanilla JS).
- 3) Melakukan setup environment React dan membuat aplikasi React menggunakan tool modern (Vite / Create React App).
- 4) Membuat dan mengelola component menggunakan JSX serta memahami struktur file aplikasi React.
- 5) Menggunakan props untuk mengirim data antar component secara satu arah (one-way data flow).
- 6) Mengelola state untuk membangun interaktivitas dasar pada aplikasi React.
- 7) Melakukan rendering list dan conditional rendering untuk menampilkan data secara dinamis.
- 8) Membangun aplikasi React sederhana yang menerapkan component, props, state, dan rendering dinamis secara terstruktur.

2. Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

- Node.js (versi LTS) -> Runtime JavaScript untuk menjalankan React dan dependency manager (npm).
- Package Manager (npm atau yarn) -> Digunakan untuk mengelola dependensi proyek React.
- React Library -> Digunakan untuk membangun antarmuka pengguna berbasis component.
- React Library -> Digunakan untuk membangun antarmuka pengguna berbasis component.
- React Library -> Digunakan untuk membangun antarmuka pengguna berbasis component.
- React Library -> Digunakan untuk membangun antarmuka pengguna berbasis component.

3. Langkah-Langkah Praktikum

Terdapat beberapa langkah-langkah untuk praktikum, yaitu:

- 1) Siapkan laptop/PC dan juga aplikasi node js yang sudah terinstal
 - Buka terminal, command prompt atau git bash.
 - Jalankan perintah berikut untuk memastikan Node.js dan npm sudah terpasang:

`node -v`

`npm -v`
- 2) Buat folder proyek
 - Tentukan lokasi penyimpanan project misal di folder pemrograman web.
 - Buka git bash pada folder yang sudah di tentukan
 - Buat folder baru untuk aplikasi React:

`mkdir react-app`

`cd react-app`
- 3) Inisialisasi aplikasi React menggunakan Vite
 - Jalankan perintah berikut: `npm create vite@latest`
 - Masukkan nama project.
 - Pilih framework React.
 - Pilih variant JavaScript atau JavaScript + SWC.
 - Pilih No pada bagian “use rolldown-vite (Experimental)?”
 - Pada bagian “Install with npm and start now?” pilih opsi “Yes”
 - Tunggu hingga proses inisialisasi selesai.
- 4) Masuk ke folder project : `cd nama-project`
- 5) Install dependency
 - Jalankan perintah: `npm install`
 - Tunggu hingga seluruh dependency berhasil diunduh.
- 6) Menjalankan development server
 - Jalankan aplikasi React dengan perintah: `npm run dev`
- 7) Mempelajari struktur folder project

Pada git bash, matikan server dengan cara `ctrl + c`, lalu Ketikan perintah berikut :

`cd nama-project`

`code .` Perintah tersebut akan otomatis membuka vscode pada folder react yang sudah dibikin.

4. Hasil dan Dokumentasi Program

API.js

```
1 import axios from "axios"; // Pastikan untuk menginstal axios dengan perintah: npm install axios
2
3 const API_URL = "/pemrograman_web/WebPro4904/Pekan-09/api"; // Sesuaikan dengan path API yang digunakan di server
4
5 // Membuat instance axios dengan konfigurasi dasar
6 const api = axios.create({
7   baseURL: API_URL,
8   headers: {
9     "Content-Type": "application/json",
10    Accept: "application/json",
11  },
12 });
13
14 // Interceptor untuk menambahkan token ke setiap request jika tersedia
15 api.interceptors.request.use(
16   (config) => {
17     const token = localStorage.getItem("token");
18     if (token) {
19       config.headers.Authorization = `Bearer ${token}`;
20     }
21     return config;
22   },
23   (error) => {
24     return Promise.reject(error);
25   },
26 );
```

```
1 // Interceptor untuk menangani error 401 Unauthorized
2 api.interceptors.response.use(
3   (response) => response,
4   (error) => {
5     if (error.response && error.response.status === 401) {
6       const requestUrl = error.config?.url || "";
7
8       if (!requestUrl.includes("/login") && !requestUrl.includes("/register")) {
9         localStorage.removeItem("token");
10        localStorage.removeItem("user");
11        window.location.href = "/login";
12      }
13    }
14    return Promise.reject(error);
15  },
16 );
17
18 // untuk mengelompokkan endpoint terkait otentikasi
19 export const authAPI = {
20   login: (email, password) => api.post("/auth/login", { email, password }),
21   register: (name, email, password, password_confirmation) =>
22     api.post("/auth/register", { name, email, password, password_confirmation }),
23   logout: () => api.post("/auth/logout"),
24   me: () => api.get("/auth/me"),
25 };
26
27 // untuk mengelompokkan endpoint terkait posts
28 export const postsAPI = {
29   getAll: (page = 1, perPage = 10) => api.get(`/post?page=${page}&per_page=${perPage}`),
30   getById: (id) => api.get(`/post/${id}`),
31 };
32
33 export default api;
34
```

Vite.config.js

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vite.dev/config/
5 export default defineConfig({
6   base: './pemrograman_web/WebPro4904/Pekan-12/dist/', // Sesuaikan dengan path yang digunakan di server
7   plugins: [react()],
8   server: {
9     proxy: {
10       '/pemrograman_web/WebPro4904/Pekan-09': { // Sesuaikan dengan path API yang digunakan di src/services/api.js
11         target: 'http://localhost',
12         changeOrigin: true,
13       },
14     },
15   },
16 })
17
```

Main.jsx

```
1 import { StrictMode } from 'react' // Pastikan untuk mengimpor StrictMode dari React
2 import { createRoot } from 'react-dom/client' // Pastikan untuk mengimpor createRoot dari react-dom/client
3 import './index.css' // Pastikan untuk membuat file index.css jika belum ada
4 import App from './App.jsx' // Pastikan untuk membuat file App.jsx jika belum ada
5
6 // Render aplikasi React ke dalam elemen dengan id 'root'
7 createRoot(document.getElementById('root')).render(
8   <StrictMode>
9     <App />
10   </StrictMode>,
11 )
12
```

Index.css

```
1 @tailwind base;
2 @tailwind components;
3 @tailwind utilities;
4
5 :root {
6   color-scheme: dark;
7 }
8
9 body {
10   font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
11 }
12
```

App.jsx

```
1 //menghubungkan semua komponen dan halaman, serta mengatur routing menggunakan React Router
2 import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
3 import { AuthProvider } from '../context/AuthContext';
4 import ProtectedRoute from '../components/ProtectedRoute';
5 import Login from '../pages/Login';
6 import PostList from '../pages/PostList';
7 import PostDetail from '../pages/PostDetail';
8 import './index.css';
9
10 // Komponen utama aplikasi yang mengatur routing dan konteks otentikasi
11 function App() {
12   return (
13     <AuthProvider>
14       <Router>
15         <Routes>
16           <Route path="/login" element={<Login />} />
17
18           <Route
19             path="/posts"
20             element={
21               <ProtectedRoute>
22                 <PostList />
23               </ProtectedRoute>
24             }
25           />
26
27           <Route
28             path="/posts/:id"
29             element={
30               <ProtectedRoute>
31                 <PostDetail />
32               </ProtectedRoute>
33             }
34           />
35
36           // Redirect root path dan wildcard path ke /posts
37           <Route path="/" element={<Navigate to="/posts" replace />} />
38
39           // Redirect wildcard path ke /posts
40           <Route path="*" element={<Navigate to="/posts" replace />} />
41         </Routes>
42       </Router>
43     </AuthProvider>
44   );
45 }
46
47 // Ekspor komponen App sebagai default export
48 export default App;
```

App.css



```
1  #root {
2    max-width: 1280px;
3    margin: 0 auto;
4    padding: 2rem;
5    text-align: center;
6  }
7
8  .logo {
9    height: 6em;
10   padding: 1.5em;
11   will-change: filter;
12   transition: filter 300ms;
13 }
14 .logo:hover {
15   filter: drop-shadow(0 0 2em #646cffaa);
16 }
17 .logo.react:hover {
18   filter: drop-shadow(0 0 2em #61dafbaa);
19 }
20
21 @keyframes logo-spin {
22   from {
23     transform: rotate(0deg);
24   }
25   to {
26     transform: rotate(360deg);
27   }
28 }
29
30 @media (prefers-reduced-motion: no-preference) {
31   a:nth-of-type(2) .logo {
32     animation: logo-spin infinite 20s linear;
33   }
34 }
35
36 .card {
37   padding: 2em;
38 }
39
40 .read-the-docs {
41   color: #888;
42 }
43
```

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46

<div className="flex items-center justify-between">
  <div className="flex items-center">
    <input
      id="remember"
      type="checkbox"
      className="w-4 h-4 border border-gray-300 rounded bg-gray-50 focus:ring-2 focus:ring-blue-500"
    />
    <label htmlFor="remember" className="ml-2 text-sm text-gray-600">
      Remember me
    </label>
  </div>
  <a href="#" className="text-sm font-medium text-blue-600 hover:text-blue-500">
    Forgot password?
  </a>
</div>

<button
  type="submit"
  disabled={loading}
  className="w-full py-3 px-4 bg-blue-600 hover:bg-blue-700 text-white font-medium rounded-lg transition-colors focus:ring-4 focus:ring-blue-300 disabled:opacity-50 disabled:cursor-not-allowed"
>
  {loading ? 'Signing in...' : 'Sign in'}
</button>

<p className="text-sm text-gray-600 text-center">
  Don't have an account yet? ( ' ' )
  <a href="#" className="font-medium text-blue-600 hover:text-blue-500">
    Sign up
  </a>
</p>
</form>
</div>

<div>
  <footer className="bg-white border-t border-gray-200 py-4">
    <p className="text-center text-sm text-gray-500">
      © 2024 Blog Posts. All rights reserved.
    </p>
  </footer>
</div>
);

```

// Export komponen Login sebagai default export agar bisa digunakan di bagian lain aplikasi

export default Login;

PostDetail

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41

//meng import React dan beberapa hook serta komponen yang diperlukan
import { useState, useEffect } from 'react';
import { useParams, Link } from 'react-router-dom';
import { postsAPI } from '../services/api';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import DefaultAvatar from '../components/DefaultAvatar';

// Komponen PostDetail yang menampilkan detail dari sebuah post berdasarkan ID yang diberikan
const PostDetail = () => {
  const { id } = useParams();
  const [post, setPost] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  // Menggunakan useEffect untuk memanggil fungsi fetchPost saat komponen pertama kali dirender atau saat ID berubah
  useEffect(() => {
    fetchPost();
  }, [id]);

  // Fungsi untuk mengambil data post dari API berdasarkan ID yang diberikan
  const fetchPost = async () => {
    // Set loading menjadi true sebelum memulai proses pengambilan data
    try {
      setLoading(true);
      const response = await postsAPI.getById(id);
      setPost(response.data.data);
      setError('');
    } catch (err) {
      // Menangani error yang mungkin terjadi selama proses pengambilan data
      if (err.response?.status === 404) {
        setError('Post tidak ditemukan.');
```

```

1 // Fungsi untuk memformat tanggal dalam format yang lebih mudah dibaca
2 const formatDate = (dateString) => {
3   if (!dateString) return '';
4   const date = new Date(dateString);
5   return date.toLocaleDateString('id-ID', {
6     day: 'numeric',
7     month: 'long',
8     year: 'numeric'
9   });
10 };
11
12 // Menampilkan loading spinner jika data masih dalam proses pengambilan
13 if (loading) {
14   return (
15     <div className="min-h-screen bg-gray-100 flex items-center justify-center">
16       <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
17     </div>
18   );
19 }
20
21 // Menampilkan pesan error jika terjadi kesalahan saat pengambilan data
22 if (error) {
23   return (
24     <div className="min-h-screen bg-gray-100 flex flex-col">
25       <Navbar />
26       <div className="flex-1 flex items-center justify-center py-8 px-4 mx-auto max-w-screen-xl lg:py-16 lg:px-6">
27         <div className="text-center py-20">
28           <p className="text-red-500 mb-4">{error}</p>
29           <Link
30             to="/posts"
31             className="text-white bg-blue-600 hover:bg-blue-700 font-medium rounded-lg text-sm px-5 py-2.5"
32           >
33             ← Kembali ke Daftar Posts
34           </Link>
35         </div>
36       </div>
37       <Footer />
38     </div>
39   );
40 }

```

PostList

```

1 //import yang berfungsi untuk mengelola state dan efek samping dalam komponen React
2 import { useState, useEffect } from 'react';
3 import { postsAPI } from '../services/api';
4 import Navbar from '../components/Navbar';
5 import Footer from '../components/Footer';
6 import PostCard from '../components/PostCard';
7
8 // Komponen PostList yang menampilkan daftar semua post yang diambil dari API
9 const PostList = () => {
10   const [posts, setPosts] = useState([]); // State untuk menyimpan daftar post yang diambil dari API
11   const [loading, setLoading] = useState(true); // State untuk menandakan apakah data masih dalam proses pengambilan
12   const [error, setError] = useState(''); // State untuk menyimpan pesan error jika terjadi kesalahan saat pengambilan data
13
14   // Menggunakan useEffect untuk memanggil fungsi fetchPosts saat komponen pertama kali dirender
15   useEffect(() => {
16     fetchPosts();
17   }, []);
18
19   // Fungsi untuk mengambil data post dari API
20   const fetchPosts = async () => {
21     // Set loading menjadi true sebelum memulai proses pengambilan data
22     try {
23       setLoading(true);
24       const response = await postsAPI.getAll(); // Memanggil fungsi getAll dari postsAPI untuk mengambil semua post
25       setPosts(response.data.data); // Menyimpan data post yang diambil ke dalam state posts
26       setError('');
27     } // Menangani error yang mungkin terjadi selama proses pengambilan data
28     catch (err) {
29       setError('Gagal memuat posts. Silakan coba lagi.');
```

5. Analisis dan Pembahasan

Setelah melakukan praktikum pada Modul ini, ada beberapa poin penting yang bisa kita bedah mengenai cara kerja React JS dalam membangun sebuah website. Di React kita memakai pendekatan deklaratif. Artinya, kita cuma perlu fokus membuat aturan main atau kondisi datanya saja (*state*), nanti React yang otomatis mengurus perubahan tampilannya di layar browser.

Saat menulis komponen tersebut, kita menggunakan JSX. Sintaks ini sangat mudah digunakan karena kita bisa menulis struktur mirip HTML langsung di dalam file JavaScript. Di praktikum ini kita menggunakan fungsi bawaan React bernama *useState*. Setiap kali kita memperbarui nilai *state* lewat fungsi pengubahnya, React akan mendeteksi perubahan tersebut dan langsung merender ulang halaman (*re-render*) secara otomatis, sehingga tampilan aplikasi kita selalu sinkron dengan data terbaru.

6. Kesimpulan

Berdasarkan praktikum pengenalan React JS yang sudah dilakukan, bisa kita simpulkan kalau library ini benar-benar mengubah cara kita membangun sebuah website menjadi jauh lebih mudah, terstruktur, dan efisien dibanding menggunakan JavaScript biasa (Vanilla JS) .

7. Link Repository

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/FadhilahAzzahra-707012500118>